

TCOS

Registration System

Backend API Specification

Version	1.0.0
Date	2026-05-16
Author	BE Engineering Team
Status	Draft — Internal
Stack	Next.js / Node.js · MongoDB · UUIDv7
Payment	PromptPay QR Only

1. Overview & Conventions

This document is the authoritative API reference for all backend routes in the TCOS Registration System. Every engineer on the BE team should treat it as source-of-truth before implementing an endpoint.

Base URL & Protocol

Protocol	HTTPS only. HTTP redirects to HTTPS in production.
Base URL	[Base URL]/v1 (all paths below are relative to this)
Format	JSON (Content-Type: application/json) except multipart upload endpoints.
Encoding	UTF-8

Authentication

Authenticated endpoints require a Bearer token in the Authorization header:

```
Authorization: Bearer <jwt_access_token>
```

Token type	JWT signed with RS256 (asymmetric). Issued at /auth/login.
Expiry	Access token: 15 min. Refresh token: 7 days (HttpOnly cookie).
Admin routes	JWT payload must contain role: "admin" claim.
OTP header	POST /auth/otp/verify requires X-OTP-Token: <token> instead of Bearer.

Standard Response Envelope

All responses use a consistent envelope:

```
// SUCCESS
{
  "success": true,
  "data": { /* resource or array */ },
  "meta": { "page": 1, "limit": 20, "total": 80 } // pagination when applicable
}

// ERROR
{
  "success": false,
  "error": {
    "code": "REGISTRATION_CLOSED",
    "message": "This activity is no longer accepting registrations.",
    "field": "activity_id" // optional, for validation errors
  }
}
```

HTTP Status Codes

200 OK	Successful GET, PATCH.
201 Created	Resource created (POST).
204 No Content	DELETE with no body.
400 Bad Request	Validation error. error.field will indicate which field.
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	JWT valid but insufficient role.
404 Not Found	Resource does not exist.
409 Conflict	Duplicate entry (e.g. already registered).
422 Un-processable	Business-logic rejection (e.g. activity full, payment mismatch).
500 Internal Error	Unexpected server error — log and alert.

Primary Key Convention

All MongoDB `_id` fields use UUIDv7. Generate with the `uuid` npm package:

```
import { v7 as uuidv7 } from "uuid";
const newId = uuidv7(); // e.g. "018f6a2e-4b3c-7abc-9def-012345678901"
```

2. Endpoint Tracking Table

Use this table to assign ownership, track implementation status, and verify test coverage during sprints. Update Status column as work progresses: Planned → In Progress → Done.

#	Endpoint	Method	Auth	Status	Priority	Domain
1	POST /auth/register	POST	None	Planned	P0	Auth
2	POST /auth/login	POST	None	Planned	P0	Auth
3	POST /auth/logout	POST	Bearer	Planned	P0	Auth
4	POST /auth/otp/send	POST	None	Planned	P1	Auth
5	POST /auth/otp/verify	POST	None	Planned	P1	Auth
6	GET /users/me	GET	Bearer	Planned	P0	Users
7	PATCH /users/me	PATCH	Bearer	Planned	P0	Users
8	GET /users/me/activities	GET	Bearer	Planned	P0	Users
9	GET /users/me/qr	GET	Bearer	Planned	P0	Users
10	GET /activities	GET	None	Planned	P0	Activities
11	GET /activities/recommended	GET	Bearer	Planned	P1	Activities
12	GET /activities/:id	GET	None	Planned	P0	Activities
13	POST /admin/activities	POST	Admin	Planned	P0	Admin
14	PATCH /admin/activities/:id	PATCH	Admin	Planned	P0	Admin
15	DELETE /admin/activities/:id	DELETE	Admin	Planned	P1	Admin
16	POST /registrations	POST	Bearer	Planned	P0	Registrations
17	GET /registrations/:id	GET	Bearer	Planned	P0	Registrations
18	POST /registrations/:id/payment	POST	Bearer	Planned	P0	Payments
19	GET /admin/registrations	GET	Admin	Planned	P0	Admin
20	PATCH /admin/registrations/:id/status	PATCH	Admin	Planned	P0	Admin
21	POST /events/scan	POST	Admin	Planned	P0	Operations
22	POST /admin/activities/:id/export	POST	Admin	Planned	P0	Admin

3. Domain: Authentication /auth

POST /auth/register — Create TCOS Account

POST**/auth/register***Create new user account***Auth:** None (public)**Flow**

1. Validate request body — all required fields present, email format valid, password $\geq$ 8 chars.
2. Check USERS collection for duplicate email. Return 409 if found.
3. Hash password with bcrypt (cost factor 12). Generate UUIDv7 for `_id`.
4. Insert new document into USERS collection.
5. Return 201 with sanitized user object (no `password_hash`).

Request

```
// POST /auth/register
// Content-Type: application/json
{
  "first_name": "Somchai",
  "last_name": "Jaidee",
  "nickname": "Golf",
  "email": "golf@example.com",
  "phone": "0812345678",
  "password": "MyPass@123",
  "gender": "Male", // "Male"|"Female"|"LGBTQ+"|"Unspecified"
  "interests": ["drama","music"] // optional, array of tag strings
}
```

Response

```
// 201 Created
{
  "success": true,
  "data": {
    "_id": "018f6a2e-4b3c-7abc-...",
    "first_name": "Somchai",
    "last_name": "Jaidee",
    "nickname": "Golf",
    "email": "golf@example.com",
    "phone": "0812345678",
    "gender": "Male",
    "interests": ["drama","music"],
    "created_at": "2026-05-16T07:00:00.000Z"
  }
}
```

Notes

- `password_hash` is NEVER returned in any response.
- If the caller also wants a session immediately after register, they must call `/auth/login` separately — or the register endpoint may optionally issue tokens (**team decision**).

POST /auth/login — Authenticate & Issue Tokens

POST**/auth/login***Authenticate user, return JWT***Auth:** None (public)

Flow

1. Validate email and password fields present.
2. Find USERS document by email. Return 401 (generic) if not found — avoid user enumeration.
3. Compare plaintext password against password_hash with `bcrypt.compare()`. Return 401 if mismatch.
4. Sign access JWT (RS256, 15 min) containing { sub: user._id, role, nickname }.
5. Sign refresh JWT (RS256, 7 days). Set as HttpOnly Secure SameSite=Strict cookie.
6. Return 200 with access token in response body.

Request

```
{
  "email": "golf@example.com",
  "password": "MyPass@123"
}
```

Response

```
// 200 OK | Set-Cookie: refresh_token=...; HttpOnly; Secure
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "user": {
      "_id": "018f6a2e-...",
      "nickname": "Golf",
      "email": "golf@example.com",
      "role": "user"
    }
  }
}
```

Notes

- Never expose a different error for wrong password vs unknown email — always return identical 401 to prevent user enumeration.

POST /auth/logout — Terminate Session

POST**/auth/logout***Invalidate refresh token*

Auth: Bearer token required

Flow

1. Verify Bearer token (can be expired — just need the sub claim).
2. Clear the refresh_token cookie by setting Max-Age=0.
3. Optionally add the JTI to a denylist in Redis/MongoDB for stateless invalidation.
4. Return 204 No Content.

Request

```
// No body required – token read from Authorization header + Cookie
```

Response

```
// 204 No Content
```

Notes

- Client should delete the access token from memory after this call.

POST /auth/otp/send — Send OTP (Optional Identity Verify)

POST**/auth/otp/send***Send OTP to phone/email*

Auth: None (public)

Flow

1. Validate phone or email field present.
2. Generate 6-digit OTP. Store hash in MongoDB with 5-minute TTL index.
3. Send via SMS gateway (phone) or transactional email service.
4. Return 200 regardless of whether the identifier exists (prevent enumeration).

Request

```
{
  "phone": "0812345678"   // or "email": "..."
}
```

Response

```
// 200 OK
{
  "success": true,
  "data": { "expires_in": 300 }
}
```

Notes

- OTP is rate-limited to 3 sends per phone per 10 minutes.

POST /auth/otp/verify — Verify OTP

POST**/auth/otp/verify***Verify submitted OTP code*

Auth: None — returns short-lived OTP verification token

Flow

1. Find stored OTP hash by identifier. Return 422 if expired or not found.
2. Compare submitted code against hash. Return 422 if mismatch.
3. Invalidate (delete) the OTP record immediately after success.
4. Issue a short-lived OTP verification token (signed JWT, 10 min) returned in response.
5. Client sends this token as X-OTP-Token header on subsequent protected actions.

Request

```
{
  "phone": "0812345678",
  "code": "483920"
}
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "otp_token": "eyJ...",
    "expires_in": 600
  }
}
```

Notes

- OTP is single-use — immediately deleted from store upon successful verification.

4. Domain: Users /users

GET /users/me — Fetch Current User Profile

GET**/users/me***Get authenticated user data*

Auth: Bearer token required

Flow

1. Decode JWT — extract sub (user_id).
2. Query USERS collection by _id. Return 404 if not found (deleted account edge case).
3. Return sanitized user document (no password_hash).

Request

```
// No body – identity from Bearer token
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "_id": "018f6a2e-...",
    "first_name": "Somchai",
    "last_name": "Jaidee",
    "nickname": "Golf",
    "email": "golf@example.com",
    "phone": "0812345678",
    "gender": "Male",
    "interests": ["drama", "music"],
    "profile_image_url": "https://cdn.example.com/avatars/xyz.jpg",
    "created_at": "2026-01-01T00:00:00.000Z"
  }
}
```

Notes

- Used by the sticky footer and dashboard to hydrate the global user state.

PATCH /users/me — Update Profile

PATCH**/users/me***Update current user profile***Auth:** Bearer token required**Flow**

1. Decode JWT — extract sub.
2. Validate only allowed fields are in the body (whitelist). Reject unknown fields with 400.
3. If email change requested: check uniqueness, require OTP re-verification via X-OTP-Token header.
4. If password change: require current_password, verify with bcrypt, hash new password.
5. Perform partial update (\$set) on USERS document. Return updated document.

Request

```
// All fields optional – send only what changes
{
  "nickname":      "Nok",
  "interests":     ["dance", "improv"],
  "profile_image_url": "https://cdn.example.com/avatars/new.jpg"
}
```

Response

```
// 200 OK
{
  "success": true,
  "data": { /* full updated user object, same shape as GET /users/me */ }
}
```

Notes

- password_hash, _id, created_at cannot be updated via this endpoint.

GET /users/me/activities — User's Registrations (My Tickets)

GET**/users/me/activities***Fetch user's registered activities***Auth:** Bearer token required**Flow**

1. Decode JWT — extract sub (user_id).
2. Query REGISTRATIONS by user_id. Apply status filter from query params if provided.
3. Lookup ACTIVITIES for each registration (_id in activity_id). Determine 'upcoming' vs 'past' by comparing activity schedule dates to now.
4. Return array of enriched registration objects.

Request

```
// Query params (optional):
// ?status=upcoming – activities with future schedule dates
// ?status=past     – activities with all schedule dates in the past
// ?reg_status=PAID – filter by registration status (PENDING|PAID|JOINED)
```

Response

```
// 200 OK
{
  "success": true,
  "data": [
    {
      "registration_id": "018f...",
      "status": "PAID",
      "registered_at": "2026-05-10T12:00:00Z",
      "group_number": null,
      "activity": {
        "_id": "018e...",
        "name": "Improv Workshop",
        "hero_image_url": "https://cdn...",
        "schedule": [{ "date": "2026-06-01", "start_time": "09:00", "end_time": "17:00", "venue": "Studio A" }]
      }
    }
  ]
}
```

GET /users/me/qr — Generate Participant QR Code**GET****/users/me/qr***Issue a short-lived signed QR for event check-in***Auth:** Bearer token required**Flow**

1. Decode JWT — extract sub (user_id).
2. Build QR payload: { user_id, iat: now, exp: now + 300 }. Note: no activity_id — QR is identity-only; admin scan resolves which activity to check the user into.
3. Sign payload with server HMAC-SHA256 secret to produce a tamper-proof string.
4. Return the signed token. Frontend renders it as a QR image client-side (e.g. qrcode.js).
5. QR expires after 300 seconds (5 minutes). Frontend must re-call this endpoint to refresh.

Request

```
// No body. Identity from Bearer token.
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "qr_token": "eyJ1c2VyX2lkIjoiMDE4Zi4uLiIsIm1hdCI6MTc0...<HMAC>",
    "expires_at": "2026-05-16T07:35:00Z",
    "expires_in": 300
  }
}
```

Notes

-
- The QR encodes user identity only, NOT a specific activity. One QR covers all their activities. Admin provides the activity context at scan time via `event_id` in the scan payload.
 - Frontend should display a countdown timer and auto-refresh the QR when `expires_in` reaches 0.
 - `qr_token` format: `base64url(JSON_payload) + '.' + HMAC_signature`.
-

5. Domain: Activities /activities

GET /activities — List Events

GET /activities

List all activities with filters

Auth: None (public)

Flow

1. Build MongoDB filter from query params.
2. Apply pagination (default page=1, limit=20). Use lean() for performance.
3. Return array of activity documents (exclude extra_questions from list view for payload size).

Request

```
// Query params (all optional):  
// ?tags=drama,music      - filter by tags (comma-separated, OR match)  
// ?is_featured=true      - featured activities only  
// ?is_registration_open=true - open activities only  
// ?page=1&limit=20       - pagination
```

Response

GET /activities/recommended — Personalized Recommendations

GET /activities/recommended

Return activities matching user interests

Auth: Bearer token required

Flow

1. Decode JWT, fetch user interests array from USERS.
2. Query ACTIVITIES where tags intersects with interests (\$in) AND is_registration_open: true.
3. Score results by overlap count, return top N ordered by score desc.
4. Fall back to featured activities if user has no interests set.

Request

```
// No body. Optional: ?limit=10
```

Response

```
// 200 OK  
{  
  "success": true,  
  "data": [ /* same shape as GET /activities items */ ]  
}
```

GET /activities/:id — Full Activity Detail

GET**/activities/:id***Return full activity document***Auth:** None (public)**Flow**

1. Validate :id is valid UUIDv7 format. Return 400 if malformed.
2. Query ACTIVITIES by _id. Return 404 if not found.
3. Return full document including schedule, extra_questions, and enrolled_count.

Request

```
// No body. :id = UUIDv7 activity ID
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "_id": "018e...",
    "name": "Improv Workshop",
    "description": "Full markdown content...",
    "hero_image_url": "https://cdn...",
    "price": 500,
    "seat_capacity": 30,
    "enrolled_count": 12,
    "tags": ["improv"],
    "is_registration_open": true,
    "is_featured": false,
    "schedule": [
      { "date": "2026-06-01", "start_time": "09:00", "end_time": "17:00", "venue": "Studio A" }
    ],
    "extra_questions": [
      {
        "question_id": "q1",
        "question_text": "What is your experience level?",
        "type": "single_choice",
        "options": ["Beginner", "Intermediate", "Advanced"],
        "is_required": true
      }
    ]
  }
}
```

Notes

- The frontend uses extra_questions to dynamically render the registration form fields.

6. Domain: Admin — Activity Management

/admin/activities

POST /admin/activities — Create Activity

POST**/admin/activities***Create a new activity document***Auth:** Bearer token (role: admin)**Flow**

1. Verify JWT role === 'admin'. Return 403 if not.
2. Validate all required fields. enrolled_count defaults to 0.
3. Generate UUIDv7 for _id. Generate question_id for each item in extra_questions.
4. Insert into ACTIVITIES collection. Return 201 with created document.

Request

```
{
  "name": "Voice Acting Intensive",
  "description": "A full-day intensive...",
  "hero_image_url": "https://cdn.example.com/img/voice.jpg",
  "price": 800,
  "seat_capacity": 20,
  "tags": ["voice", "intensive"],
  "is_registration_open": false,
  "is_featured": true,
  "schedule": [
    { "date": "2026-07-15", "start_time": "10:00", "end_time": "18:00", "venue": "Room B" }
  ],
  "extra_questions": [
    { "question_text": "Do you have prior VA experience?", "type": "single_choice", "options":
      ["Yes", "No"], "is_required": true }
  ]
}
```

Response

```
// 201 Created
{
  "success": true,
  "data": { /* full activity document with generated _id and question_ids */ }
}
```

Notes

- enrolled_count is always initialized to 0 on creation — never set by the client.

PATCH /admin/activities/:id — Update Activity

PATCH**/admin/activities/:id***Update activity fields***Auth:** Bearer token (role: admin)**Flow**

1. Verify admin role.
2. Validate :id. Find activity. Return 404 if not found.
3. Whitelist allowed update fields. enrolled_count cannot be updated directly via this route.
4. Perform \$set update. Return updated document.

Request

```
// All fields optional
{
  "is_registration_open": true,
  "seat_capacity":      25,
  "is_featured":        false
}
```

Response

```
// 200 OK
{
  "success": true,
  "data": { /* full updated activity document */ }
}
```

Notes

- Use this to open/close registration, adjust capacity, or correct details before launch.

DELETE /admin/activities/:id — Delete Activity

DELETE**/admin/activities/:id***Soft-delete or hard-delete activity***Auth:** Bearer token (role: admin)**Flow**

1. Verify admin role.
2. Check if any REGISTRATIONS with status PAID or JOINED exist for this activity. If yes, return 409 — cannot delete activity with active registrations.
3. If only PENDING registrations exist, cancel them (update status to CANCELLED).
4. Soft-delete: add deleted_at: Date field to activity document (recommended) OR hard delete.
5. Return 204.

Request

```
// No body.
```

Response

```
// 204 No Content
```

Notes

- Recommended: soft-delete via `deleted_at` field and filter deleted activities out of public queries.

7. Domain: Registrations /registrations

POST /registrations — Initiate Registration

POST

/registrations

Create a new registration
(supports guest + new-account
flow)

Auth: Bearer token (existing user) OR inline new-user payload (new user)

Flow

1. CASE A — Existing user: Decode Bearer JWT, extract user_id.
2. CASE B — New user (no Bearer token): Payload contains new_user object. Follow the same logic as POST /auth/register to create account atomically, then use new_id as user_id. Return access token alongside registration response.
3. Validate activity_id exists and is_registration_open === true. Return 422 if closed.
4. Check seat_capacity vs enrolled_count. Return 422 if full.
5. Check for duplicate registration (same user_id + activity_id with status != CANCELLED). Return 409 if found.
6. Validate custom_answers against activity extra_questions — ensure all is_required questions answered.
7. Insert REGISTRATIONS document with status: PENDING. Return 201.

Request

```
// CASE A — Existing user (Bearer token in header)
{
  "activity_id": "018e...",
  "custom_answers": [
    { "question_id": "q1", "answer": "Beginner" }
  ]
}

// CASE B — New user (no Bearer token)
{
  "activity_id": "018e...",
  "custom_answers": [...],
  "new_user": {
    "first_name": "Malee",
    "last_name": "Sooksai",
    "nickname": "May",
    "email": "may@example.com",
    "phone": "0898765432",
    "password": "TempPass@1",
    "gender": "Female"
  }
}
```

Response

```
// 201 Created
{
  "success": true,
  "data": {
    "registration_id": "018f...",
    "status": "PENDING",
    "activity_id": "018e...",
  }
}
```

```
"registered_at": "2026-05-16T07:30:00Z",
"activity": { "name": "Improv Workshop", "price": 500 },
"access_token": "eyJ..." // CASE B only – new user token
}
}
```

Notes

- Seat count is NOT incremented here — only incremented upon PAID confirmation to avoid ghost reservations.
- For free activities (price == 0), status is set directly to PAID at creation and enrolled_count is incremented immediately.

GET /registrations/:id — Get Registration Detail

GET**/registrations/:id***Fetch a single registration*

Auth: Bearer token (owner or admin)

Flow

1. Decode JWT. Find registration by :id.
2. Verify requesting user is the owner (user_id matches) or has admin role. Return 403 otherwise.
3. Return registration with joined activity snapshot.

Request

```
// No body.
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "_id": "018f...",
    "user_id": "018e...",
    "activity_id": "018d...",
    "status": "PENDING",
    "group_number": null,
    "registered_at": "2026-05-16T07:30:00Z",
    "custom_answers": [{ "question_id": "q1", "answer": "Beginner" }],
    "activity": { /* activity name, price, schedule */ }
  }
}
```

8. Domain: Payments /registrations/:id/payment

POST /registrations/:id/payment — Submit & Verify PromptPay Slip

POST /registrations/:id/payment

Upload slip, verify QR, confirm payment

Auth: Bearer token (registration owner)

Flow

1. Verify Bearer token. Confirm registration belongs to requesting user. Return 403 otherwise.
2. Confirm registration status is PENDING. Return 422 if already PAID.
3. Accept multipart/form-data. Read slip image into memory buffer — DO NOT write to disk or S3.
4. Pass buffer to in-memory QR decoder (e.g. zxing-js or jimp + jsqr). Extract QR string.
5. Parse QR string — validate PromptPay EMV format. Extract transaction amount and reference.
6. Compare extracted amount against activity.price. Return 422 AMOUNT_MISMATCH if different.
7. Check PAYMENTS collection for duplicate QR data (same promptpay_qr_data). Return 409 if already used.
8. Insert PAYMENTS document (status: VERIFIED). Discard buffer immediately (GC eligible).
9. Update REGISTRATIONS status to PAID via atomic \$set.
10. Increment activity enrolled_count by 1 via \$inc on ACTIVITIES document.
11. Return 200 with payment confirmation.

Request

```
// Content-Type: multipart/form-data
// Field: slip (image file - JPEG or PNG)
// The image is processed in-memory and never persisted.
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "payment_id": "018g...",
    "registration_id": "018f...",
    "status": "VERIFIED",
    "amount": 500,
    "verified_at": "2026-05-16T07:35:00Z"
  }
}

// 422 - Amount mismatch
{
  "success": false,
  "error": { "code": "AMOUNT_MISMATCH", "message": "Slip amount (฿300) does not match required amount (฿500)." }
}
```

Notes

- Image buffer is explicitly nulled after QR extraction to make it GC-eligible — no S3/R2 write occurs.
- The \$inc on enrolled_count must be atomic to prevent race conditions at capacity.

- If QR decode fails (blurry image), return 422 QR_UNREADABLE — user must resubmit.

9. Domain: Admin Operations /admin & /events

GET /admin/registrations — List All Registrations

GET /admin/registrations

Admin view of all registrations

Auth: Bearer token (role: admin)

Flow

1. Verify admin role.
2. Support query filters: ?activity_id=, ?status=PENDING|PAID|JOINED, ?page=, ?limit=.
3. Join with user and activity documents (MongoDB \$lookup or separate queries).
4. Return paginated list.

Request

```
// Query params:  
// ?activity_id=018e...    - filter by activity  
// ?status=PAID           - filter by registration status  
// ?page=1&limit=50
```

Response

```
// 200 OK  
{  
  "success": true,  
  "meta": { "page": 1, "limit": 50, "total": 120 },  
  "data": [  
    {  
      "_id": "018f...",  
      "status": "PAID",  
      "registered_at": "2026-05-16T07:30:00Z",  
      "group_number": null,  
      "user": { "full_name": "Somchai Jaidee", "email": "golf@example.com", "phone": "081..." },  
      "activity": { "name": "Improv Workshop" },  
      "custom_answers": [...]  
    }  
  ]  
}
```

PATCH /admin/registrations/:id/status — Manually Update Registration Status

PATCH /admin/registrations/:id/status

Override registration status (admin)

Auth: Bearer token (role: admin)

Flow

1. Verify admin role.
2. Find registration by :id.
3. Validate status transition is allowed (e.g. PENDING → PAID, PAID → JOINED).
4. Update status. If transitioning to PAID from PENDING manually, also \$inc enrolled_count on activity.
5. Return updated registration.

Request

```
{
  "status":      "PAID",
  "group_number": "A-03"  // optional, relevant for JOINED
}
```

Response

```
// 200 OK
{
  "success": true,
  "data": { /* updated registration object */ }
}
```

POST /events/scan — Scan QR Code at Entry

POST /events/scan

Validate entry QR, update status to JOINED

Auth: Bearer token (role: admin or staff)

Flow

1. Verify admin/staff role.
2. Receive qr_payload string from scanner device.
3. Verify HMAC signature on payload. Return 422 INVALID_QR if tampered or expired.
4. Parse registration_id from payload. Find REGISTRATION document.
5. Confirm status === PAID. Return 422 if already JOINED (duplicate scan) or PENDING.
6. Atomically update status to JOINED
7. Return user info and group data for display on scanner device.

Request

```
{
  "qr_payload":  "eyJyZWdpY3RyYXRpb25fawQiOiIwMThmLi4uIn0.HMAC",
  "event_id":    ""
}
```

```
}

```

Response

```
// 200 OK — Entry Granted
{
  "success": true,
  "data": {
    "registration_id": "018f...",
    "status": "JOINED",
    "group_number": "B-07",
    "user": {
      "full_name": "Somchai Jaidee",
      "nickname": "Golf"
    },
    "activity_name": "Improv Workshop"
  }
}

// 422 — Already scanned
{
  "success": false,
  "error": { "code": "ALREADY_JOINED", "message": "This ticket was already scanned at 09:14." }
}
```

Notes

- Duplicate scan protection is critical — guard against volunteers accidentally scanning twice.

POST /events/scan — Scan Participant QR at Entry

POST	/events/scan	Verify participant QR + event membership, update status to <i>JOINED</i>
-------------	---------------------	--

Auth: Bearer token (role: admin or staff)

Flow

1. Verify admin/staff role.
2. Receive qr_token (from participant QR) and event_id from the scanner payload.
3. Verify HMAC signature of qr_token. Return 422 INVALID_QR if signature mismatch.
4. Check exp claim inside qr_token. Return 422 QR_EXPIRED if past expiry (5-min window).
5. Extract user_id from the decoded payload.
6. Query REGISTRATIONS: find a record where user_id matches AND activity_id === event_id. Return 404 if no such registration exists (user not enrolled in this event).
7. Check registration status === PAID. Return 422 ALREADY_JOINED if status is already JOINED. Return 422 if PENDING (not paid).
8. Atomically \$set status to JOINED.
9. Return user info and group data for display on scanner device.

Request

```
{
  "qr_token": "eyJ1c2VyX2lkIjoiMDE4Zi4uLiIsImIhdCI6MTc0...<HMAC>",
  "event_id": "018e..." // activity UUIDv7 — identifies which event to check against
}
```

Response

```
// 200 OK — Entry Granted
{
  "success": true,
  "data": {
    "registration_id": "018f...",
    "status": "JOINED",
    "user": {
      "full_name": "Somchai Jaidee",
      "nickname": "Golf"
    },
    "activity_name": "Improv Workshop"
  }
}

// 422 — QR expired
{
  "success": false,
  "error": { "code": "QR_EXPIRED", "message": "QR code has expired. Ask participant to refresh." }
}

// 404 — Not enrolled in this event
{
  "success": false,
  "error": { "code": "NOT_ENROLLED", "message": "This participant is not registered for this event." }
}

// 422 — Already scanned
{
  "success": false,
  "error": { "code": "ALREADY_JOINED", "message": "This ticket was already scanned at 09:14." }
}
```

Notes

- The QR carries user identity only. The event_id in the body is what binds the check-in to a specific activity — allowing the same QR to be used at multiple events the user is enrolled in.
- Duplicate scan protection is critical — guard against staff accidentally scanning twice.
- QR_EXPIRED should prompt the scanner UI to ask the participant to refresh their QR screen.

POST /admin/activities/:id/export — Export to Google Sheets

POST /admin/activities/:id/export

Trigger ETL pipeline to Google Sheets

Auth: Bearer token (role: admin)

Flow

1. Verify admin role. Validate activity exists.
2. **EXTRACT:** Query all REGISTRATIONS for activity_id with status PAID or JOINED. Join with USERS for name/contact fields.
3. **TRANSFORM:** Flatten custom_answers by question_id → column mapping. Build row array.

4. LOAD: Use Google Sheets API (service account). Create or clear target sheet tab named after activity name + export timestamp.
5. Write header row (dynamic — built from extra_questions) then data rows in a single batchUpdate call.
6. Return Sheets URL and row count on success.

Request

```
// No body required. :id = activity UUIDv7
```

Response

```
// 200 OK
{
  "success": true,
  "data": {
    "sheet_url": "https://docs.google.com/spreadsheets/d/SPREADSHEET_ID/edit#gid=0",
    "rows_exported": 58,
    "exported_at": "2026-05-16T08:00:00Z"
  }
}
```

Notes

- Use a Google Service Account with the Sheets API. Store credentials in environment variables (never committed to repo).
- The ETL runs synchronously for <= 500 rows. Queue via a background job (e.g. BullMQ) for larger datasets.
- Sheet columns: Registration ID | Name | Nickname | Email | Phone | Status | Group | Registered At | [dynamic extra_question columns...]

10. Error Code Reference

Error Code	HTTP	Description
VALIDATION_ERROR	400	Missing or malformed request field.
DUPLICATE_EMAIL	409	Email already registered.
DUPLICATE_REGISTRATION	409	User already registered for this activity.
DUPLICATE_PAYMENT	409	This payment slip QR has already been used.
INVALID_CREDENTIALS	401	Email or password incorrect.
TOKEN_EXPIRED	401	JWT access token has expired.
FORBIDDEN	403	Insufficient role for this operation.
NOT_FOUND	404	Requested resource does not exist.
REGISTRATION_CLOSED	422	Activity is not accepting registrations.
ACTIVITY_FULL	422	Activity has reached seat capacity.
ALREADY_PAID	422	Registration is already in PAID state.
AMOUNT_MISMATCH	422	Slip amount does not match required price.
QR_UNREADABLE	422	Could not decode QR from uploaded slip.
INVALID_QR	422	QR payload signature is invalid or expired.
ALREADY_JOINED	422	Ticket was already used for entry.
INTERNAL_ERROR	500	Unexpected server error. Check logs.

11. Architectural Constraints (Summary)

Database	MongoDB. All _id fields are UUIDv7 (uuid npm package, v7).
Primary Keys	UUIDv7 across all collections — time-sorted, globally unique.
Payment	PromptPay QR only. Slip processed in-memory; image never written to disk or object storage.
Slip Verification	In-memory QR decode (zxing-js / jsqr). Buffer nulled immediately after extraction.
Real-Time Counts	enrolled_count uses MongoDB \$inc for atomic increments. No COUNT queries on read.
Seats Check	Seat availability checked at registration time using enrolled_count vs seat_capacity.
Export	Hybrid ETL to Google Sheets via Service Account. BullMQ for > 500 rows.
Auth	RS256 JWT. Access: 15 min. Refresh: 7 days HttpOnly cookie.
Free Activities	price === 0 → status set to PAID immediately; enrolled_count incremented at registration.